

Proyecto intermodular de Prometeo

Memoria de base de datos para EñicaFix S.L

Ismael Medardo Torres Becerra

Noa Maximilian Tóth-Égető

Introducción	1
1. Análisis de los datos del sistema	2
1.1. Empresas (Clientes)	2
1.2. Catálogo de Servicios	2
1.3. Equipo Técnico (Personal)	2
1.4. Solicitudes de Contacto (Leads / Formulario Web)	2
1.5. Tickets de Soporte (Incidencias y Etiquetado)	3
1.6. Facturación y Registro de Trabajos	3
Ciclo de vida del negocio:	3
1. Fase de Captación (Entrada de Datos)	3
2. Fase de Conversión (Estructuración)	3
3. Fase Operativa (Gestión de Servicios y Soporte)	3
4. Fase de Salida y Cierre (Facturación y Control)	4
2. Diseño de la base de datos	4
2.1. Modelo entidad-relación	4
2.1.1. Entidades y Atributos Principales	4
2.1.2. Relaciones y Cardinalidades	5
2.1.3. Diagrama Entidad-Relación	6
2.2. Modelo Relación	6
2.2.1. Definición de Tablas y Atributos	6
2.2.2. Tablas Intermedias (Resolución de relaciones N:M)	8
2.2.3. Reglas de Integridad Referencial	8
2.2.4. Verificación de Formas Normales	8
3. Creación de la base de datos	9
3.1. Decisiones técnicas y motor de base de datos	9
3.2. Script SQL de creación (DDL)	10
4. Inserción y gestión de datos	13
4.1. Consideraciones de la inserción	13
4.2. Script SQL de Inserción (INSERT)	13
5. Consultas y administración básica	17
5.1. Consultas con filtros (Gestión de prioridades)	17
5.2. JOIN entre tablas (Visión global del soporte)	17
5.3. Listados de información (Control de Facturación)	18
5.4. Consultas de búsqueda (Buscador de incidencias)	18
5.5. Consulta avanzada con M:N (Detalle de Factura)	18
6. Tareas básicas de administración	19
6.1. Copias de seguridad de la base de datos (Backups)	19
6.2. Exportación de datos	19
6.3. Gestión básica de usuarios y permisos	20
7. Conclusión	20

Introducción

El presente documento detalla el diseño, desarrollo y administración de la base de datos para **EficaFix S.L.**, una empresa dedicada al soporte técnico y la prestación de servicios informáticos. Este proyecto tiene como objetivo aplicar de forma práctica los conocimientos adquiridos en el módulo de Bases de Datos del primer curso del ciclo de ASIR.

Para que EficaFix pueda operar de manera eficiente, necesita centralizar su información. El sistema diseñado permite gestionar de forma integral su cartera de clientes (empresas), el catálogo de servicios ofrecidos, la plantilla de técnicos, el seguimiento en tiempo real de las incidencias (tickets) y la facturación básica.

A lo largo de esta memoria se documentará todo el ciclo de vida de la base de datos:

- **Diseño:** Creación del Modelo Entidad-Relación (conceptual) y su paso al Modelo Relacional (lógico).
- **Implementación:** Código de creación (DDL) y poblado de datos (DML) utilizando el motor PostgreSQL.
- **Explotación y Mantenimiento:** Elaboración de consultas SQL para el día a día de la empresa y definición de las tareas básicas de administración, copias de seguridad y seguridad del sistema.

1. Análisis de los datos del sistema

Para que el sistema de Eficafix funcione correctamente, la base de datos debe centralizar tanto la captación de clientes como la operativa técnica y administrativa. Las entidades identificadas son:

1.1. Empresas (Clientes)

- **Qué información se guarda:** Nombre comercial, CIF/NIF, correo corporativo, teléfono y ubicación.
- **Por qué es necesaria:** Para identificar legalmente a los clientes, gestionar la comunicación oficial y tener los datos necesarios para la facturación.
- **Cómo se relaciona:** Es el núcleo del sistema; las facturas y los tickets de soporte siempre están vinculados a una empresa cliente.

1.2. Catálogo de Servicios

- **Qué información se guarda:** Nombre del servicio (Mantenimiento, Redes, Ciberseguridad, Cloud, etc.), descripción técnica y tarifa plana mensual.
- **Por qué es necesaria:** Para estandarizar la oferta comercial de Eficafix y automatizar el cálculo de costes en facturas y presupuestos.
- **Cómo se relaciona:** Se vincula con las solicitudes de contacto (interés) y con las facturas emitidas.

1.3. Equipo Técnico (Personal)

- **Qué información se guarda:** Nombre, correo interno y responsabilidades específicas:
 - **Ismael:** Soporte técnico, atención al cliente, seguridad, administración de servidores y etiquetado/organización de tareas.
 - **Noa:** Desarrollo de código, administración de servidores y etiquetado de incidencias técnicas.
- **Por qué es necesaria:** Para gestionar la asignación de trabajo según la especialidad de cada miembro del equipo.
- **Cómo se relaciona:** Cada ticket o tarea del sistema tiene un técnico responsable asignado.

1.4. Solicitudes de Contacto (Leads / Formulario Web)

- **Qué información se guarda:** Nombre del interesado, email, nombre de su empresa, servicio por el que consulta, mensaje y fecha/hora.
- **Por qué es necesaria:** Para almacenar y gestionar todos los mensajes que llegan a través del archivo contacto.html de la web. Permite convertir un usuario anónimo en un cliente potencial.
- **Cómo se relaciona:** Si la gestión comercial tiene éxito, los datos de esta tabla se utilizan para dar de alta a la entidad en la tabla "Empresas".

1.5. Tickets de Soporte (Incidencias y Etiquetado)

- **Qué información se guarda:** Fecha, asunto, descripción, estado (Abierto/Cerrado), técnico asignado y **etiquetas (tags)**.
- **Por qué es necesaria:** Para asegurar la respuesta en menos de 24 horas. Las etiquetas permiten clasificar el problema (ej: "Prioridad Alta", "Bug de Código", "Fallo de Red") para que Ismael o Noa actúen rápido.
- **Cómo se relaciona:** Conecta al Cliente con el Técnico encargado de solucionar su incidencia.

1.6. Facturación y Registro de Trabajos

- **Qué información se guarda:** Número de factura, fecha, concepto (servicio prestado), base imponible, IVA y estado de cobro (Pagado/Pendiente).
- **Por qué es necesaria:** Para llevar el control contable de la empresa y asegurar que cada intervención o servicio mensual quede registrado y cobrado.
- **Cómo se relaciona:** Se genera cruzando los datos de la **Empresa** con los precios del **Catálogo de Servicios**.

Ciclo de vida del negocio:

1. Fase de Captación (Entrada de Datos)

El ciclo comienza en el mundo exterior (la web). Aquí la base de datos actúa como un receptor de oportunidades.

- **Punto de entrada:** El formulario contacto.html.
- **Acción:** Se crea un registro en la entidad Solicitudes de Contacto.
- **Dato clave:** El "Servicio por el que consulta" se coteja con el Catálogo de Servicios para entender qué busca el cliente potencial.

2. Fase de Conversión (Estructuración)

Si la gestión comercial tiene éxito, la información "volátil" del lead se convierte en información "maestra".

- **Transformación:** Los datos de la solicitud migran para dar de alta a una nueva entidad en la tabla Empresas.
- **Integridad:** En este punto, el sistema asigna el CIF/NIF y la ubicación, convirtiendo al interesado en un cliente legal vinculado al sistema de facturación.

3. Fase Operativa (Gestión de Servicios y Soporte)

Una vez que el cliente existe, la base de datos gestiona el día a día de Eficafix. Aquí interactúan tres ejes: Cliente, Servicio y Equipo Técnico.

- **Asignación de Trabajo:** Se generan Tickets de Soporte. La base de datos cruza el problema del cliente con el perfil del Equipo Técnico (ej. si es un "Bug de Código", el sistema permite identificar que Noa es el responsable).
- **Categorización:** El uso de etiquetas (tags) en los tickets permite filtrar la carga de trabajo y asegurar el cumplimiento de la promesa de respuesta en menos de 24 horas.

4. Fase de Salida y Cierre (Facturación y Control)

El ciclo de cada interacción o mes de servicio termina con la monetización y el registro histórico.

- **Generación de Valor:** La entidad Facturación extrae los precios del Catálogo de Servicios y los vincula a la Empresa.
- **Registro de Trabajos:** No solo se guarda cuánto se cobra, sino qué se hizo. Se crea un histórico donde cada factura está respaldada por los servicios prestados o los tickets resueltos.

2. Diseño de la base de datos

En esta fase se detalla la estructura lógica y conceptual de la base de datos necesaria para soportar las operaciones de **EficaFix S.L.** El diseño se ha elaborado garantizando la integridad de los datos, evitando la redundancia y asegurando la escalabilidad del sistema.

2.1. Modelo entidad-relación

El modelo Entidad-Relación (E/R) abstrae los requisitos del sistema en entidades principales (los objetos del negocio), sus atributos (las características de esos objetos) y las relaciones que existen entre ellos.

A continuación, se definen los elementos que componen nuestro modelo conceptual:

2.1.1. Entidades y Atributos Principales

- **EMPRESA (Cliente):** Almacena los datos fiscales y de contacto de los clientes B2B.
 - **Atributos:** PK id_empresa, nombre_comercial, cif_nif, correo, telefono, direccion, fecha_alta, activo.
- **CATÁLOGO_SERVICIO:** Representa la oferta comercial y los servicios prestados por la empresa.
 - **Atributos:** PK id_servicio, nombre_servicio, descripcion, tarifa_mensual, activo.
- **TÉCNICO (Personal):** Registra a los empleados de EficaFix (ej. Ismael, Noa) encargados de resolver incidencias.
 - **Atributos:** PK id_tecnico, nombre, correo_interno, activo.
- **ESPECIALIDAD:** Entidad dependiente para gestionar las áreas de conocimiento de cada técnico, cumpliendo con las reglas de normalización (evitando atributos multivaluados).
 - **Atributos:** PK id_especialidad, descripcion, area.
- **SOLICITUD_CONTACTO (Leads):** Entidad que registra los mensajes entrantes desde el formulario de la página web.
 - **Atributos:** PK id_solicitud, nombre_interesado, email, nombre_empresa, mensaje, fecha_hora, estado.
- **TICKET (Soporte):** Gestiona las incidencias o peticiones de soporte técnico.
 - **Atributos:** PK id_ticket, fecha_creacion, asunto, descripcion, estado, prioridad, fecha_cierre.

- TAG (Etiquetas): Permite categorizar los tickets de manera flexible (ej. "Bug", "Urgente", "Redes").
 - **Atributos:** PK id_tag, nombre_tag, color.
- FACTURA: Entidad administrativa para la facturación de los servicios prestados a las empresas.
 - **Atributos:** PK id_factura, numero_factura, fecha_emision, concepto, base_imponible, iva_porcentaje, total, estado_cobro.

2.1.2. Relaciones y Cardinalidades

Las interacciones entre las entidades se definen mediante la siguiente cardinalidad (Mínima, Máxima):

1. Abre (EMPRESA - TICKET) [1:N]:
 - Una empresa puede abrir múltiples tickets a lo largo del tiempo (1, n).
 - Un ticket pertenece única y exclusivamente a una empresa (1, 1).
2. Atiende (TÉCNICO - TICKET) [1:N]:
 - Un técnico tiene asignados uno o varios tickets para su resolución (1, n).
 - Un ticket es atendido por un único técnico responsable (1, 1).
3. Tiene (TÉCNICO - ESPECIALIDAD) [1:N]:
 - Un técnico puede tener una o varias especialidades técnicas (1, n).
 - Una especialidad registrada en este contexto pertenece a un técnico (1, 1).
4. Recibe (EMPRESA - FACTURA) [1:N]:
 - Una empresa recibe múltiples facturas (mensuales o puntuales) (1, n).
 - Una factura se emite a nombre de una sola empresa (1, 1).
5. Consulta (CATÁLOGO_SERVICIO - SOLICITUD_CONTACTO) [1:N]:
 - Un servicio del catálogo puede ser el motivo de consulta de múltiples leads (0, n).
 - Una solicitud de contacto hace referencia a un único servicio de interés (1, 1).
6. Convierte en (SOLICITUD_CONTACTO - EMPRESA) [0..1 : 1]:
 - Una solicitud de contacto puede convertirse (o no) en una empresa cliente (0, 1).

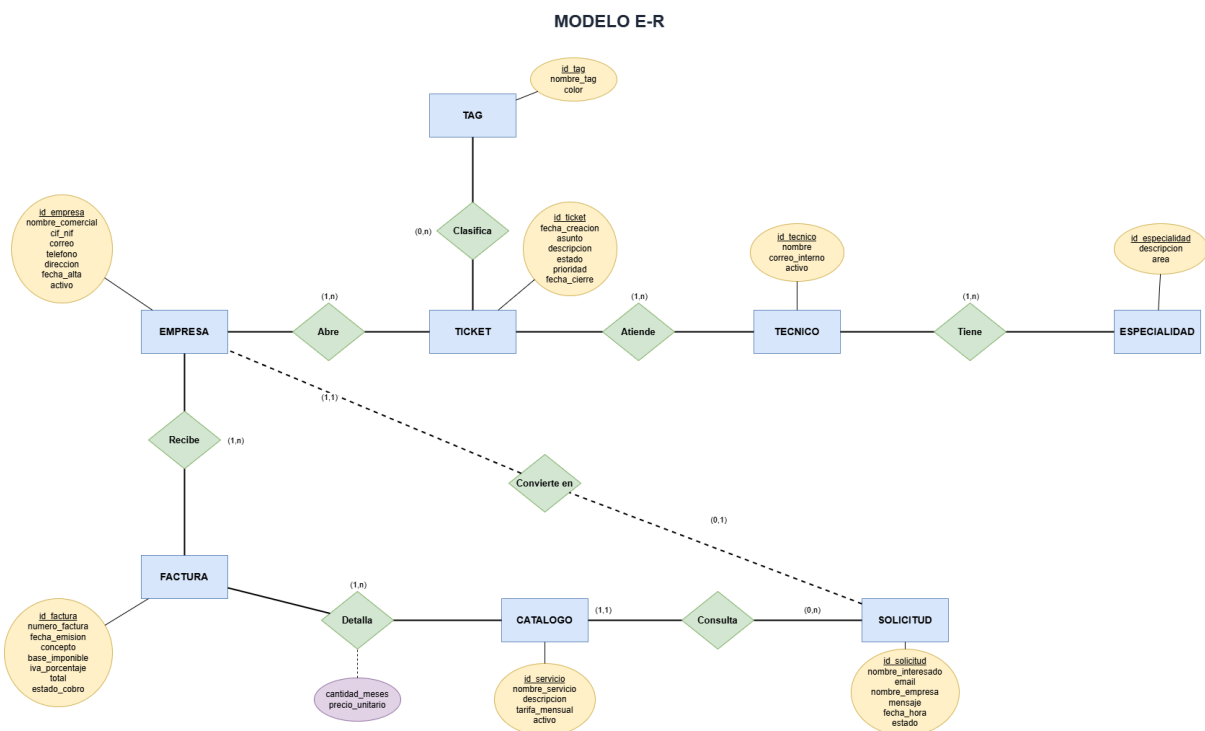
7. Detalle (FACTURA - CATÁLOGO_SERVICIO) [N:M]:

- Una factura puede incluir varios servicios (1, n).
- Un servicio puede aparecer facturado en múltiples facturas (0, n).
- Atributos de esta relación: cantidad_meses y precio_unitario.
(Nota: Esta relación compleja generará una tabla intermedia en el modelo relacional).

8. Clasifica (TICKET - TAG) [N:M]:

- Un ticket puede tener varias etiquetas asignadas (1, n).
- Una etiqueta puede agrupar múltiples tickets (0, n).

2.1.3. Diagrama Entidad-Relación



2.2. Modelo Relacion

El modelo relacional traduce el diseño conceptual a una estructura de tablas interconectadas. En este paso, las relaciones se materializan mediante el uso de Claves Foráneas (FK), y las relaciones Muchos a Muchos (N:M) se descomponen en tablas asociativas para cumplir con las reglas de normalización.

2.2.1. Definición de Tablas y Atributos

A continuación se detallan las tablas que componen la base de datos de **EficaFix S.L.**:

1. EMPRESA (Clientes)

- id_empresa (INT, PK, AUTO_INCREMENT)
- nombre_comercial (VARCHAR(100), NOT NULL)
- cif_nif (VARCHAR(15), UNIQUE, NOT NULL)
- correo (VARCHAR(100))
- telefono (VARCHAR(20))
- direccion (TEXT)
- fecha_alta (DATE, DEFAULT CURRENT_DATE)
- activo (BOOLEAN, DEFAULT TRUE)

2. TECNICO (Personal)

- id_tecnico (INT, PK, AUTO_INCREMENT)
- nombre (VARCHAR(100), NOT NULL)
- correo_interno (VARCHAR(100), UNIQUE)
- activo (BOOLEAN, DEFAULT TRUE)

3. ESPECIALIDAD (Áreas técnicas)

- id_especialidad (INT, PK, AUTO_INCREMENT)
- id_tecnico (INT, FK → TECNICO)
- descripcion (VARCHAR(100))
- area (VARCHAR(50))

4. TICKET (Gestión de incidencias)

- id_ticket (INT, PK, AUTO_INCREMENT)
- id_empresa (INT, FK → EMPRESA)
- id_tecnico (INT, FK → TECNICO)
- fecha_creacion (DATETIME, DEFAULT CURRENT_TIMESTAMP)
- asunto (VARCHAR(150))
- descripcion (TEXT)
- estado (ENUM('Abierto', 'En Proceso', 'Resuelto', 'Cerrado'))
- prioridad (ENUM('Baja', 'Media', 'Alta', 'Crítica'))
- fecha_cierre (DATETIME, NULL)

5. CATALOGO_SERVICIO (Oferta comercial)

- id_servicio (INT, PK, AUTO_INCREMENT)
- nombre_servicio (VARCHAR(100), NOT NULL)
- descripcion (TEXT)
- tarifa_mensual (DECIMAL(10,2))
- activo (BOOLEAN, DEFAULT TRUE)

6. SOLICITUD_CONTACTO (Leads de la web)

- id_solicitud (INT, PK, AUTO_INCREMENT)
- id_servicio (INT, FK → CATALOGO_SERVICIO, NULL)
- nombre_interesado (VARCHAR(100))
- email (VARCHAR(100))
- nombre_empresa (VARCHAR(100))
- mensaje (TEXT)
- fecha_hora (DATETIME, DEFAULT CURRENT_TIMESTAMP)
- estado (ENUM('Pendiente', 'Contactado', 'Convertido'))

7. FACTURA (Administración)

- id_factura (INT, PK, AUTO_INCREMENT)

- id_empresa (INT, FK → EMPRESA)
- numero_factura (VARCHAR(30), UNIQUE)
- fecha_emision (DATE)
- base_imponible (DECIMAL(10,2))
- iva_porcentaje (DECIMAL(4,2), DEFAULT 21.00)
- total (DECIMAL(10,2))
- estado_cobro (ENUM('Pendiente', 'Pagado', 'Vencido'))

2.2.2. Tablas Intermedias (Resolución de relaciones N:M)

Para las relaciones Muchos a Muchos identificadas en el modelo E/R, se han creado las siguientes tablas:

TICKET_TAG: Vincula los tickets con sus etiquetas.

- id_ticket (INT, FK → TICKET)
- id_tag (INT, FK → TAG)
- PK Compuesta: (id_ticket, id_tag)

FACTURA_SERVICIO: Detalla qué servicios incluye cada factura y a qué precio en ese momento.

- id_factura (INT, FK → FACTURA)
- id_servicio (INT, FK → CATALOGO_SERVICIO)
- cantidad_meses (INT, DEFAULT 1)
- precio_unitario (DECIMAL(10,2))
- PK Compuesta: (id_factura, id_servicio)

2.2.3. Reglas de Integridad Referencial

Para mantener la coherencia de los datos:

1. **Eliminación:** Se aplicará ON DELETE RESTRICT en la mayoría de las relaciones para evitar borrar empresas o técnicos que tengan tickets o facturas asociadas.
2. **Actualización:** Se aplicará ON UPDATE CASCADE para asegurar que los cambios en los identificadores se propaguen correctamente.

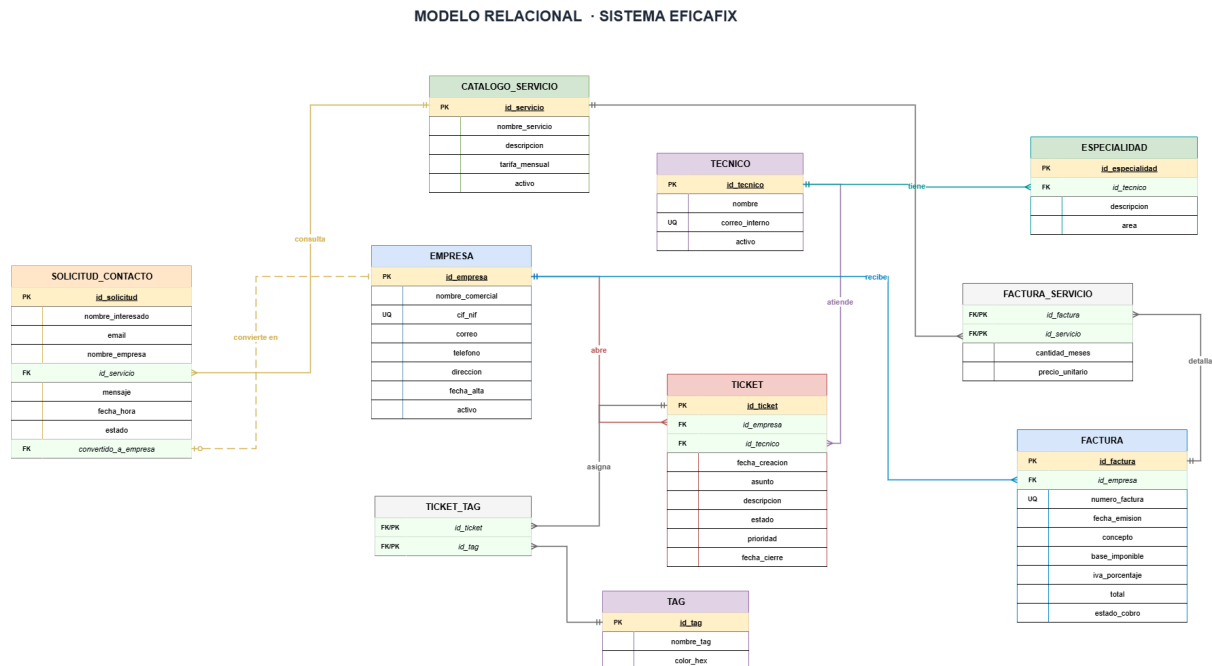
2.2.4. Verificación de Formas Normales

1FN  — No hay atributos multivaluados. Las responsabilidades de cada técnico se separaron en la tabla ESPECIALIDAD. Los tags de los tickets están en TICKET_TAG.

2FN  — Todas las tablas con PK simple: todos los atributos dependen funcionalmente de la PK completa. En las tablas pivote (TICKET_TAG, FACTURA_SERVICIO), los atributos adicionales (precio_unitario, cantidad_meses) dependen de toda la clave compuesta, no solo de una parte.

3FN  — Ningún atributo no-clave depende transitivamente de otro atributo no-clave. El campo total en FACTURA podría calcularse en

aplicación, pero al almacenarlo se documenta como campo derivado para rendimiento.



3. Creación de la base de datos

En este apartado se presenta el código DDL (*Data Definition Language*) necesario para implementar físicamente la base de datos de **EficaFix S.L.** El script crea el esquema, define las tablas con sus respectivas Claves Primarias (PK) y Foráneas (FK), y establece las restricciones de integridad necesarias.

3.1. Decisiones técnicas y motor de base de datos

Para la implementación de este proyecto, se ha elegido el motor PostgreSQL. Por este motivo, el código SQL se ha adaptado a su dialecto y características específicas, aplicando las siguientes reglas de diseño:

- **Autoincrementales:** Se utiliza el tipo de dato nativo SERIAL para la generación automática de las Claves Primarias (PK) en lugar del tradicional AUTO_INCREMENT.
- **Gestión de Fechas:** Se ha utilizado DATE para fechas exactas (como la emisión de facturas) y TIMESTAMP para registrar el momento exacto (fecha y hora) de la creación de tickets o la entrada de leads web.
- **Protección de Historial (No eliminación):** Para garantizar la integridad del sistema, los registros históricos no se eliminan físicamente. En su lugar:
 - Se utiliza la restricción ON DELETE RESTRICT en las Claves Foráneas de TICKET y FACTURA. Esto impide por completo que

un administrador borre por error a un cliente o a un técnico que tenga incidencias o facturación asociada.

- Se implementa un borrado lógico en entidades principales (Empresa, Técnico, Catálogo) mediante el campo activo BOOLEAN.
- Los tickets finalizados simplemente cambian su campo estado a 'Cerrado', manteniéndose en la base de datos por motivos de auditoría.
- Restricciones de Dominio (CHECK): En lugar de utilizar tipos de datos ENUM externos, se ha optado por restricciones CHECK directamente en la creación de la tabla (ej. CHECK (estado IN ('Abierto', 'Resuelto', ...))), lo que asegura que solo se introduzcan valores permitidos de forma nativa y eficiente.
- Actualización en Cascada: Se han añadido reglas ON UPDATE CASCADE y ON DELETE CASCADE en las tablas intermedias y dependientes para garantizar que, si se realiza un cambio legítimo en un ID, la base de datos mantenga su coherencia automáticamente.

3.2. Script SQL de creación (DDL)

A continuación se muestra el código completo empleado para la creación de la base de datos en PostgreSQL:

```
-- =====
-- PROYECTO INTERMODULAR: EFICAFIX S.L.
-- SCRIPTS DE CREACIÓN DE BASE DE DATOS (POSTGRESQL)
-- =====

-- 1. Tabla EMPRESA
CREATE TABLE EMPRESA (
    id_empresa SERIAL PRIMARY KEY,
    nombre_comercial VARCHAR(100) NOT NULL,
    cif_nif VARCHAR(15) NOT NULL UNIQUE,
    correo VARCHAR(100),
    telefono VARCHAR(20),
    direccion TEXT,
    fecha_alta DATE DEFAULT CURRENT_DATE,
    activo BOOLEAN DEFAULT TRUE
);

-- 2. Tabla TECNICO
CREATE TABLE TECNICO (
    id_tecnico SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo_interno VARCHAR(100) UNIQUE NOT NULL,
    activo BOOLEAN DEFAULT TRUE
);
```

-- 3. Tabla ESPECIALIDAD

```
CREATE TABLE ESPECIALIDAD (  
    id_especialidad SERIAL PRIMARY KEY,  
    id_tecnico INT NOT NULL,  
    descripcion VARCHAR(100) NOT NULL,  
    area VARCHAR(50),  
    CONSTRAINT fk_especialidad_tecnico  
        FOREIGN KEY (id_tecnico) REFERENCES TECNICO(id_tecnico)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

-- 4. Tabla CATALOGO_SERVICIO

```
CREATE TABLE CATALOGO_SERVICIO (  
    id_servicio SERIAL PRIMARY KEY,  
    nombre_servicio VARCHAR(100) NOT NULL,  
    descripcion TEXT,  
    tarifa_mensual DECIMAL(10, 2) NOT NULL,  
    activo BOOLEAN DEFAULT TRUE  
);
```

-- 5. Tabla SOLICITUD_CONTACTO

```
CREATE TABLE SOLICITUD_CONTACTO (  
    id_solicitud SERIAL PRIMARY KEY,  
    id_servicio INT,  
    nombre_interesado VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    nombre_empresa VARCHAR(100),  
    mensaje TEXT,  
    fecha_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    estado VARCHAR(20) DEFAULT 'Pendiente'  
        CHECK (estado IN ('Pendiente', 'Contactado', 'Convertido')),  
    CONSTRAINT fk_solicitud_servicio  
        FOREIGN KEY (id_servicio) REFERENCES  
        CATALOGO_SERVICIO(id_servicio)  
        ON DELETE SET NULL ON UPDATE CASCADE  
);
```

-- 6. Tabla TICKET

```
CREATE TABLE TICKET (  
    id_ticket SERIAL PRIMARY KEY,  
    id_empresa INT NOT NULL,  
    id_tecnico INT NOT NULL,  
    asunto VARCHAR(150) NOT NULL,  
    descripcion TEXT,  
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    estado VARCHAR(20) DEFAULT 'Abierto'  
        CHECK (estado IN ('Abierto', 'En Proceso', 'Resuelto', 'Cerrado')),
```

```
prioridad VARCHAR(20) DEFAULT 'Media'
    CHECK (prioridad IN ('Baja', 'Media', 'Alta', 'Crítica')),
fecha_cierre TIMESTAMP,
CONSTRAINT fk_ticket_empresa
    FOREIGN KEY (id_empresa) REFERENCES EMPRESA(id_empresa)
    ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_ticket_tecnico
    FOREIGN KEY (id_tecnico) REFERENCES TECNICO(id_tecnico)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

-- 7. Tabla TAG
CREATE TABLE TAG (
    id_tag SERIAL PRIMARY KEY,
    nombre_tag VARCHAR(50) NOT NULL UNIQUE,
    color VARCHAR(7) DEFAULT '#3498db'
);

-- 8. Tabla FACTURA
CREATE TABLE FACTURA (
    id_factura SERIAL PRIMARY KEY,
    id_empresa INT NOT NULL,
    numero_factura VARCHAR(30) NOT NULL UNIQUE,
    fecha_emision DATE NOT NULL,
    base_imponible DECIMAL(10, 2) NOT NULL,
    iva_porcentaje DECIMAL(4, 2) DEFAULT 21.00,
    total DECIMAL(10, 2) NOT NULL,
    estado_cobro VARCHAR(20) DEFAULT 'Pendiente'
    CHECK (estado_cobro IN ('Pendiente', 'Pagado', 'Vencido')),
CONSTRAINT fk_factura_empresa
    FOREIGN KEY (id_empresa) REFERENCES EMPRESA(id_empresa)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

-- 9. Tabla Intermedia TICKET_TAG
CREATE TABLE TICKET_TAG (
    id_ticket INT NOT NULL,
    id_tag INT NOT NULL,
    PRIMARY KEY (id_ticket, id_tag),
    CONSTRAINT fk_tt_ticket FOREIGN KEY (id_ticket) REFERENCES
TICKET(id_ticket) ON DELETE CASCADE,
    CONSTRAINT fk_tt_tag FOREIGN KEY (id_tag) REFERENCES TAG(id_tag)
    ON DELETE CASCADE
);

-- 10. Tabla Intermedia FACTURA_SERVICIO
CREATE TABLE FACTURA_SERVICIO (
    id_factura INT NOT NULL,
```

```
id_servicio INT NOT NULL,  
cantidad_meses INT DEFAULT 1,  
precio_unitario DECIMAL(10, 2) NOT NULL,  
PRIMARY KEY (id_factura, id_servicio),  
CONSTRAINT fk_fs_factura FOREIGN KEY (id_factura) REFERENCES  
FACTURA(id_factura) ON DELETE CASCADE,  
CONSTRAINT fk_fs_servicio FOREIGN KEY (id_servicio) REFERENCES  
CATALOGO_SERVICIO(id_servicio) ON DELETE RESTRICT  
);
```

4. Inserción y gestión de datos

Una vez creada la estructura lógica y física de la base de datos, el siguiente paso es poblar con datos de prueba (*Mock data*). Este proceso, realizado mediante lenguaje de manipulación de datos (DML), es fundamental para verificar que las restricciones de integridad funcionan correctamente y para poder ejecutar consultas de prueba que validen el sistema.

4.1. Consideraciones de la inserción

Para evitar errores de integridad referencial (violación de Claves Foráneas), las inserciones se han realizado siguiendo un orden jerárquico estricto:

1. Consultas y entidades independientes: EMPRESA, TECNICO, CATALOGO_SERVICIO, TAG.
2. Entidades de primer nivel de dependencia: ESPECIALIDAD, SOLICITUD_CONTACTO, FACTURA, TICKET.
3. Entidades asociativas (N:M): TICKET_TAG, FACTURA_SERVICIO.

4.2. Script SQL de Inserción (INSERT)

A continuación se detalla el script utilizado para poblar la base de datos en PostgreSQL:

```
-- =====  
  
-- PROYECTO INTERMODULAR: EFICAFIX S.L.  
  
-- SCRIPT DE INSERCIÓN DE DATOS (DML)  
  
-- =====  
  
-- 1. Inserción de Técnicos (Personal de EficaFix)  
  
INSERT INTO TECNICO (nombre, correo_interno) VALUES  
  
('Ismael', 'ismael@eficafix.es'),  
  
('Noa', 'noa@eficafix.es'),
```

```
('Carlos Sánchez', 'csanchez@eficafix.es');
```

```
-- 2. Inserción de Especialidades de los técnicos
```

```
-- (1=Ismael, 2=Noa, 3=Carlos)
```

```
INSERT INTO ESPECIALIDAD (id_tecnico, descripcion, area) VALUES
```

```
(1, 'Administración de Servidores Linux y Windows', 'Sistemas'),
```

```
(1, 'Gestión de Redes Cisco', 'Redes'),
```

```
(2, 'Ciberseguridad y Pentesting', 'Seguridad'),
```

```
(2, 'Soporte Nivel 2 y Resolución de Bugs', 'Software'),
```

```
(3, 'Mantenimiento y Reparación de Equipos', 'Hardware');
```

```
-- 3. Inserción del Catálogo de Servicios
```

```
INSERT INTO CATALOGO_SERVICIO (nombre_servicio, descripcion,  
tarifa_mensual) VALUES
```

```
('Mantenimiento Básico', 'Soporte técnico 8x5 y revisión mensual', 150.00),
```

```
('Mantenimiento Premium', 'Soporte 24/7, monitorización y backups', 350.00),
```

```
('Auditoría de Seguridad', 'Análisis de vulnerabilidades puntual', 500.00),
```

```
('Instalación de Redes', 'Diseño y cableado estructurado por oficina', 800.00);
```

```
-- 4. Inserción de Empresas (Clientes)
```

```
INSERT INTO EMPRESA (nombre_comercial, cif_nif, correo, telefono, direccion)  
VALUES
```

```
('TechNova S.L.', 'B12345678', 'admin@technova.es', '600111222', 'Calle Innovación  
15, Madrid'),
```

```
('Abogados Ruiz & Cía', 'B87654321', 'info@ruizabogados.com', '600333444',  
'Avenida de la Ley 40, Valencia'),
```

```
('Clínica Dental Sonrisas', 'B11223344', 'contacto@clinicasonrisas.es', '600555666',  
'Plaza Mayor 3, Sevilla');
```


-- 5. Inserción de Etiquetas (Tags para los tickets)

INSERT INTO TAG (nombre_tag, color) VALUES

('Urgente', '#e74c3c'),

('Hardware', '#f1c40f'),

('Software', '#3498db'),

('Redes', '#2ecc71');

-- 6. Inserción de Solicitudes de Contacto (Leads web)

INSERT INTO SOLICITUD_CONTACTO (id_servicio, nombre_interesado, email, nombre_empresa, mensaje, estado) VALUES

(2, 'Laura Gómez', 'lgomez@startupxy.com', 'Startup XY', 'Necesitamos presupuesto para soporte 24/7', 'Pendiente'),

(3, 'Pedro Martínez', 'pmartinez@gestoria.es', 'Gestoría Sur', 'Queremos auditar nuestros servidores', 'Contactado'),

(NULL, 'María López', 'maria.l@gmail.com', NULL, '¿Reparáis ordenadores a particulares?', 'Convertido');

-- 7. Inserción de Tickets de Soporte

INSERT INTO TICKET (id_empresa, id_tecnico, asunto, descripcion, estado, prioridad) VALUES

(1, 2, 'Fallo en la pasarela de pago', 'La web no procesa los pagos con tarjeta.', 'En Proceso', 'Crítica'),

(2, 1, 'No hay acceso a la carpeta compartida', 'Los abogados no pueden acceder al NAS de facturación.', 'Abierto', 'Alta'),

(3, 3, 'Impresora no responde', 'La impresora de recepción se ha quedado sin IP.', 'Resuelto', 'Baja'),

(1, 1, 'Caída del servidor de correo', 'No entran ni salen emails desde esta mañana.', 'Cerrado', 'Crítica');

-- 8. Asignación de Tags a los Tickets (Tabla intermedia)

-- Ticket 1: Urgente, Software

```
INSERT INTO TICKET_TAG (id_ticket, id_tag) VALUES (1, 1), (1, 3);
```

-- Ticket 2: Urgente, Redes

```
INSERT INTO TICKET_TAG (id_ticket, id_tag) VALUES (2, 1), (2, 4);
```

-- Ticket 3: Hardware

```
INSERT INTO TICKET_TAG (id_ticket, id_tag) VALUES (3, 2);
```

-- 9. Inserción de Facturas

```
INSERT INTO FACTURA (id_empresa, numero_factura, fecha_emision,  
base_imponible, total, estado_cobro) VALUES
```

```
(1, 'F-2026-001', '2026-04-01', 350.00, 423.50, 'Pagado'),
```

```
(2, 'F-2026-002', '2026-04-05', 150.00, 181.50, 'Pendiente'),
```

```
(3, 'F-2026-003', '2026-04-10', 950.00, 1149.50, 'Vencido');
```

-- 10. Detalle de Facturas (Tabla intermedia FACTURA_SERVICIO)

-- Factura 1 incluye Mantenimiento Premium

```
INSERT INTO FACTURA_SERVICIO (id_factura, id_servicio, cantidad_meses,  
precio_unitario) VALUES
```

```
(1, 2, 1, 350.00);
```

-- Factura 2 incluye Mantenimiento Básico

```
INSERT INTO FACTURA_SERVICIO (id_factura, id_servicio, cantidad_meses,  
precio_unitario) VALUES
```

```
(2, 1, 1, 150.00);
```

-- Factura 3 incluye Mantenimiento Básico + Instalación de Redes

```
INSERT INTO FACTURA_SERVICIO (id_factura, id_servicio, cantidad_meses,  
precio_unitario) VALUES
```

```
(3, 1, 1, 150.00),
```

```
(3, 4, 1, 800.00);
```

5. Consultas y administración básica

Una vez la base de datos está estructurada y poblada, el sistema debe ser capaz de extraer información útil para la gestión diaria de **EficaFix S.L.** A continuación, se presentan diversas consultas SQL (DML) diseñadas para la administración técnica y comercial de la empresa.

5.1. Consultas con filtros (Gestión de prioridades)

Esta consulta es fundamental para el equipo técnico. Permite filtrar los tickets para mostrar únicamente aquellos que requieren atención urgente y que aún no han sido resueltos.

Objetivo: Listar los tickets abiertos o en proceso que tengan una prioridad "Alta" o "Crítica".

```
SELECT
  id_ticket,
  asunto,
  estado,
  prioridad,
  fecha_creacion
FROM TICKET
WHERE estado IN ('Abierto', 'En Proceso')
AND prioridad IN ('Alta', 'Crítica')
ORDER BY prioridad DESC, fecha_creacion ASC;
```

Output Messages Notifications

id_ticket [PK] integer	asunto character varying (150)	estado character varying (20)	prioridad character varying (20)	fecha_creacion timestamp without time zone
1	Fallo en la pasarela de pago	En Proceso	Crítica	2026-04-24 12:08:24.116683
2	No hay acceso a la carpeta comparti...	Abierto	Alta	2026-04-24 12:08:24.116683

5.2. JOIN entre tablas (Visión global del soporte)

Como los datos están normalizados, la información de un ticket por sí sola solo muestra números (IDs). Utilizando la cláusula JOIN, cruzamos la tabla TICKET con EMPRESA y TECNICO para obtener un informe legible.

Objetivo: Mostrar un panel de control con el nombre del cliente, el técnico asignado y el estado de cada incidencia.

```
SELECT
  t.id_ticket,
  e.nombre_comercial AS cliente,
  tec.nombre AS tecnico_responsable,
  t.asunto,
  t.estado
FROM TICKET t
JOIN EMPRESA e ON t.id_empresa = e.id_empresa
JOIN TECNICO tec ON t.id_tecnico = tec.id_tecnico;
```

Output Messages Notifications

id_ticket integer	cliente character varying (100)	tecnico_responsable character varying (100)	asunto character varying (150)	estado character varying (20)
1	TechNova S.L.	Noa	Fallo en la pasarela de pago	En Proceso
2	Abogados Ruiz & Cía	Ismael	No hay acceso a la carpeta comparti...	Abierto
3	Clinica Dental Sonrisas	Carlos Sánchez	Impresora no responde	Resuelto
4	TechNova S.L.	Ismael	Caída del servidor de correo	Cerrado

5.3. Listados de información (Control de Facturación)

Para la administración económica de la empresa, es vital llevar un control de los cobros. Esta consulta junta datos de facturas y clientes para que el departamento de contabilidad sepa a quién reclamar pagos.

Objetivo: Obtener un listado de las facturas que están pendientes de pago o vencidas, incluyendo el nombre de la empresa y el teléfono de contacto.

```
SELECT
    f.numero_factura,
    e.nombre_comercial,
    e.telefono,
    f.total,
    f.fecha_emision,
    f.estado_cobro
FROM FACTURA f
JOIN EMPRESA e ON f.id_empresa = e.id_empresa
WHERE f.estado_cobro IN ('Pendiente', 'Vencido')
ORDER BY f.fecha_emision ASC;
```

numero_factura	nombre_comercial	telefono	total	fecha_emision	estado_cobro
F-2026-002	Abogados Ruiz & Cía	600333444	181.50	2026-04-05	Pendiente
F-2026-003	Clinica Dental Sonrisas	600555666	1149.50	2026-04-10	Vencido

5.4. Consultas de búsqueda (Buscador de incidencias)

En el día a día, un técnico puede necesitar buscar si una incidencia similar ya ha ocurrido en el pasado. Para ello, utilizamos el operador ILIKE (específico de PostgreSQL para búsquedas insensibles a mayúsculas/minúsculas) combinándolo con comodines (%).

Objetivo: Buscar cualquier ticket que menciona la palabra "correo" o "email" en su asunto o descripción.

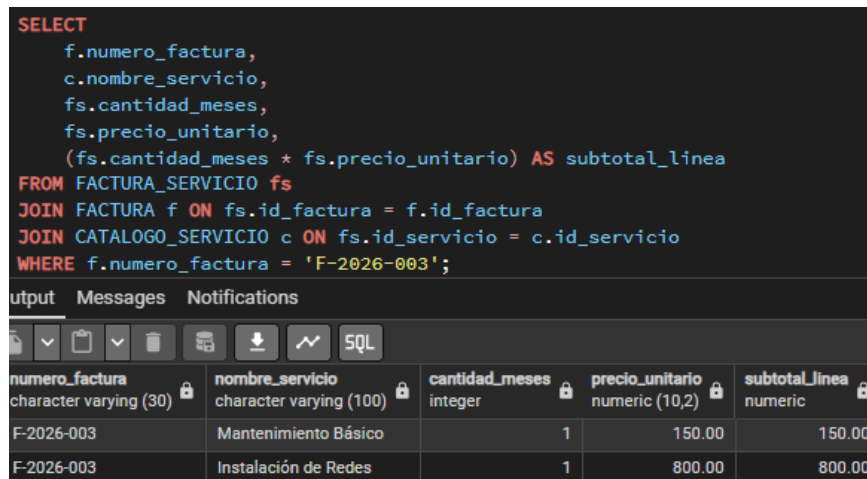
```
SELECT
    id_ticket,
    asunto,
    descripcion,
    estado
FROM TICKET
WHERE asunto ILIKE '%correo%'
    OR asunto ILIKE '%email%'
    OR descripcion ILIKE '%correo%'
    OR descripcion ILIKE '%email%';
```

id_ticket	asunto	descripcion	estado
4	Caída del servidor de corr...	No entran ni salen emails desde esta mañan...	Cerrado

5.5. Consulta avanzada con M:N (Detalle de Factura)

Para demostrar el dominio de las tablas intermedias, esta consulta extrae la relación Muchos a Muchos entre las facturas y el catálogo de servicios.

Objetivo: Listar el desglose exacto de los servicios que se han cobrado en la factura 'F-2026-003'.



The screenshot shows a SQL query in a dark-themed editor. The query selects invoice details and service information for a specific invoice number. Below the query, the results are displayed in a table with five columns: numero_factura, nombre_servicio, cantidad_meses, precio_unitario, and subtotal_linea. The results show two rows of data for invoice F-2026-003.

```
SELECT
    f.numero_factura,
    c.nombre_servicio,
    fs.cantidad_meses,
    fs.precio_unitario,
    (fs.cantidad_meses * fs.precio_unitario) AS subtotal_linea
FROM FACTURA f
JOIN FACTURA fs ON fs.id_factura = f.id_factura
JOIN CATALOGO_SERVICIO c ON fs.id_servicio = c.id_servicio
WHERE f.numero_factura = 'F-2026-003';
```

numero_factura	nombre_servicio	cantidad_meses	precio_unitario	subtotal_linea
F-2026-003	Mantenimiento Básico	1	150.00	150.00
F-2026-003	Instalación de Redes	1	800.00	800.00

6. Tareas básicas de administración

Para garantizar la seguridad, disponibilidad e integridad de la información de **EficaFix S.L.**, es fundamental llevar a cabo tareas periódicas de mantenimiento y administración sobre el gestor de bases de datos (PostgreSQL). A continuación, se detallan los procedimientos básicos.

6.1. Copias de seguridad de la base de datos (Backups)

La realización de copias de seguridad es la medida de contingencia principal ante posibles desastres (borrados accidentales, fallos de hardware o ciberataques). En PostgreSQL, la herramienta estándar de consola para realizar copias de seguridad lógicas es `pg_dump`.

Ejemplo de ejecución: Para generar un script SQL completo con toda la estructura y los datos de nuestra base de datos, ejecutaríamos el siguiente comando en la terminal del servidor:

```
pg_dump -U postgres -d eficazfix_db -f /backups/eficafix_backup_completo.sql
```

Este comando genera un archivo `.sql` que se puede restaurar fácilmente en otro servidor en caso de migración o desastre utilizando la herramienta `psql`.

6.2. Exportación de datos

En muchas ocasiones, departamentos como contabilidad o dirección necesitan extraer datos específicos para analizarlos en hojas de cálculo (como Excel o Calc) sin tener que acceder directamente a la base de datos. Para ello, PostgreSQL permite exportar el resultado de las consultas a formatos estándar como **CSV**.

Ejemplo de ejecución: Exportación de la tabla de facturas a un archivo CSV, incluyendo los nombres de las columnas en la cabecera:

COPY FACTURA TO '/tmp/listado_facturas.csv' DELIMITER ',' CSV HEADER;

6.3. Gestión básica de usuarios y permisos

Por motivos de seguridad, se debe aplicar el **Principio de Menor Privilegio**. No todos los empleados ni aplicaciones deben conectarse a la base de datos utilizando el superusuario (postgres). Es necesario crear roles específicos limitando qué tablas pueden ver o modificar.

Ejemplo de creación de usuarios y asignación de permisos:

1. **Creación de un usuario para la aplicación web:**

```
CREATE USER app_eficafix WITH PASSWORD 'AppS3cur3P@ss';
```

2. **Asignación de permisos limitados:** Se le otorgan permisos únicamente para leer, insertar y actualizar datos, pero **no** para borrar registros ni para alterar la estructura (DDL) de las tablas.

```
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO app_eficafix;
```

```
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO app_eficafix;
```

3. **Creación de un usuario de solo lectura (Auditoría/Contabilidad):**

```
CREATE USER contabilidad WITH PASSWORD 'Contab2026';
```

```
GRANT SELECT ON FACTURA, EMPRESA, FACTURA_SERVICIO TO contabilidad;
```

De esta forma, el usuario "contabilidad" podrá ver las facturas y los clientes, pero el sistema le denegará el acceso si intenta ver los tickets técnicos o si intenta modificar algún dato.

7. Conclusión

El desarrollo de este proyecto para **EficaFix S.L.** ha permitido aplicar de forma práctica todo el ciclo de vida de una base de datos, desde su diseño conceptual (Modelo E/R) hasta su implementación física y administración en **PostgreSQL**.

A lo largo del trabajo hemos construido un sistema robusto, normalizado y preparado para un entorno real. La aplicación de restricciones de integridad para proteger el historial, la creación de consultas SQL para agilizar el día a día y la planificación de tareas de seguridad (como backups y gestión de permisos) demuestran que una base de datos bien diseñada no es solo un almacén de información, sino el motor fundamental que permite a una empresa operar de forma eficiente y segura.